

מה יקרה אם יהיו עשרות או מאות משתמשים?

מערכת המבוססת על Next.js ו-Supabase מסוגלת להתמודד בקלות עם עשרות משתמשים במקביל עבור פעולות סטנדרטיות כמו צפייה בכספות ויצירת פרויקטים עם קבצים קטנים.

נקודת התורפה בסקייל הנוכחי אינה מסד הנתונים עצמו, אלא מגבלות תוכנית ה-Free של Supabase ומנגנון הצ'אט. בתוכנית החינמית יש כ-1GB לאחסון קבצים, 5GB יציאת רשת בחודש, ועד 200 חיבורי Realtime במקביל.

הצ'אט כבר מבוסס על Supabase Realtime — חיבור WebSocket שדוחף עדכון ללקוח כשיש הודעה חדשה. בנוסף נשאר Polling כל 8 שניות כגיבוי. לכן 100 משתמשים שפעילים בצ'אט בו-זמנית ייצרו בערך 12.5 בקשות HTTP בשנייה, וגם 100 חיבורי WebSocket. מגבלת 200 החיבורים תתמלא לפני שהשרת ייחנק ממספר הבקשות. כל בקשת GET עדיין שולפת את כל היסטוריית ההודעות של הפרויקט.

יצירת כספות עם קבצים גדולים אינה מעמיסה על Postgres. במסד נשמרות רק כתובות URL; הקבצים עצמם עולים ישירות ל-Supabase Storage. המגבלה האמיתית היא מכסת האחסון וה-egress, לא מספר השורות בטבלאות.

אילו שאילות למסד הנתונים עלולות להיות כבדות?

1. שליפת היסטוריית הודעות (Chat): השאילתה בנתיב `/api/chat` שולפת את כל ההודעות

מטבלת `ChatMessage` עם JOIN לטבלת `user` של השולח, בלי הגבלת כמות. ככל שהשיחה מתארכת, השליפה כבדה יותר. בנוסף, כל GET בודק קודם את כל שדות הפרויקט (`select *` על `DeliveryProject`).

2. טעינת הדאשבורד של הפריילנסר: שאילתה ששולפת את כל רשומות `DeliveryProject` של פריילנסר מסוים, כולל JOIN לטבלאות הלקוחות (`user`) והנכסים (`Asset`), עלולה להיות כבדה אם יש היסטוריה של מאות פרויקטים. הדאשבורד גם שולף את כל המשתמשים האחרים (מזהה, שם, אימייל) עבור בחירת לקוח — שאילתה שגדלה עם מספר המשתמשים במערכת, לא רק עם מספר הכספות.

האם צריך אינדקסים במסד הנתונים?

בהחלט. קיימים מפתחות זרים, אבל Postgres לא יוצר עליהם אינדקס אוטומטית, ובסכמה הנוכחית אין אינדקסים משניים על עמודות הסינון. כדי לתמוך בסקייל:

- בטבלת ChatMessage: אינדקס על העמודה projectId, כי כל שאילתת צ'אט מסננת הודעות לפי מזהה הפרויקט (WHERE projectId = X). כדאי גם אינדקס על createdAt למיון לפי זמן, ועל senderId.
- בטבלת DeliveryProject: אינדקסים על freelancerId ועל clientId, כדי לזרז את טעינת הדאשבורד שמסנן פרויקטים לפי משתמש.

איך נמנעים מטעינה מיותרת של מידע?

- **שימוש ב-React Server Components:** בארכיטקטורת Next.js App Router אנו שולפים נתונים בצד השרת בדאשבורד ובדף הכספת (/p/[projectId]) ומרנדרים אותם ל-HTML לפני השליחה ללקוח. כך לא נשלחים לדפדפן שדות רגילים כמו סיסמאות.
- **שליפת קבצים לפי דרישה:** קובץ המקור הכבד לא נטען בפתיחת הפרויקט. הוא נמסר רק בלחיצה על הורדה ואחרי בדיקת הרשאה (בעלים, או לקוח ששילם). קבצי stock מקומיים מוזרמים כ-File Stream מהדיסק; קבצים שהועלו מופנים ל-URL חתום ב-Supabase Storage ל-60 שניות, כך ש-Next.js לא מעביר את הקובץ הכבד דרך השרת.
- **העלאה ישירות ל-Storage:** הדפדפן מעלה ל-signed URL, בלי לעבור דרך API של Next.js (מגבלת גוף בקשה של כ-4.5MB ב-Vercel). תמונות מכווצות ל-WebP בצד הלקוח לפני ההעלאה.
- הערה: חילוץ zip של דמו חי כן נטען לזיכרון תהליך Next.js בעת השמירה, ואז נכתב כקבצים נפרדים ל-Storage.

האם יש שימוש נכון ב-pagination?

בגרסת ה-MVP הנוכחית אין שימוש ב-Pagination. אם היסטוריית הצ'אט ארוכה, כל ההודעות נשלפות בבת אחת. גם רשימת הפרויקטים בדאשבורד נשלפת במלואה. בסקייל גבוה חובה לממש Pagination או Infinite Scroll:

- בצ'אט: שליפת 50 ההודעות האחרונות, וטעינת הודעות ישנות יותר רק כשהמשתמש גולל למעלה.
- בדאשבורד פרויקטים: פיצול רשימת הפרויקטים לעמודים.

האם יש הפרדה נכונה בין צד לקוח וצד שרת?

קיימת הפרדה המבוססת על ארכיטקטורת Next.js:

- **צד השרת (API Routes ו-Server Actions):** מחזיק בגישה בלעדית למסד הנתונים באמצעות Service Role, מבצע את כל בדיקות ההרשאה (האם הפרויקט שולם, האם הלקוח מורשה),

מנהל את הסשן מול JWT של NextAuth, ומאמת חתימות של Webhooks מול PayPal.

- **צד הלקוח (Client Components):** אחראי על ממשק המשתמש, תצוגת אנימציות, מנוי Realtime לצ'אט ולרענון הדאשבורד, ו-Polling כל 8 שניות כגיבוי. אין בדפדפן מפתח Service Role או לוגיקת תשלום. כן יש מפתח anon הציבורי ל-Realtime, דחיסת תמונות, והעלאה ישירה ל-Storage. שגיאות מוצגות כטקסט בטופס, לא כ-Toasts.

אילו מגבלות קיימות במוצר הנוכחי?

1. **אין pagination, וחסרים אינדקסים** — היסטוריית צ'אט ורשימת פרויקטים נשלפות במלואן, בלי אינדקסים משניים על עמודות סינון.
2. **צ'אט לא ממוטב לגמרי:** Realtime כבר פעיל, אבל נשאר Polling כל 8 שניות, וכל רענון שולף את כל ההודעות. בדאשבורד מנוי Realtime על הטבלאות DeliveryProject, Asset ו-User בלי סינון שורה, כך ששינוי בטבלה מרענן כל דאשבורד פתוח.
3. **אין Rate Limiting** על העלאות קבצים, הודעות צ'אט ו-checkout.
4. **מגבלות Supabase Free:** 1GB לאחסון קבצים, עד 50MB לקובץ (באפליקציה עד 30MB לווידאו ו-20MB לתמונה), ו-5GB egress. העלאות סטנדרטיות (לא TUS) לקבצים גדולים מ-6MB פחות אמינות לפי המלצת Supabase.
5. **שריד מקומי בלבד:** קבצי stock נשמרים ב-public/stock, ויש נתיבי fallback ישנים תחת storage/. העלאות חדשות של משתמשים כבר ב-Supabase Storage, ולכן אחסון מקומי אינו חוסם עוד Horizontal Scaling.

מה הייתם משפרים בגרסה עתידית כדי לתמוך בסקייל גדול יותר?

1. **שיפור הצ'אט וה-Realtime:** הסרת מנגנון ה-Polling, מימוש Pagination או Infinite Scroll להיסטוריית הודעות, וסינון מנויי Realtime בדאשבורד לפי פרויקט או משתמש — כדי שהשרת ידחוף נתונים רק כשיש עדכון רלוונטי.
 2. **אינדקסים ו-pagination:** הוספת האינדקסים שפורטו למעלה, ופיצול רשימת הפרויקטים בדאשבורד לעמודים.
 3. **מימוש Rate Limiting:** הגבלת כמות בקשות API מאותו משתמש או כתובת IP — למשל עד 10 העלאות קבצים לדקה או 20 הודעות צ'אט לדקה — כדי להגן על השרת מהצפות ומתקפות שעלולות להפיל אותו.
 4. **העלאות גדולות:** מעבר ל-TUS (resumable uploads) עבור וידאו מעל 6MB, לאמינות טובה יותר ברשת לא יציבה.
- לא נדרשת הגירה ל-Amazon S3 או ל-Supabase Storage: שמירת נכסי המקור והתצוגות המקדימות כבר מתבצעת ב-Supabase Storage דרך signed URLs.